

Credit Card Fraud Detection Using Autoencoders

SUBMITTED BY: TEAM ALPHA NEURONS

Introduction

This assignment aims to create and test out a fraud detection system, using Autoencoders, specifically simple Autoencoders and Variational Autoencoders (VAE). The process used for this is anomaly detection. We will be using real credit card fraud data from 2013 of European customers to train our model. Through these unsupervised models we aim for it to learn the patterns of normal transaction and reconstruct non-fraud events, thereby identifying outliers and detecting fraudulent transactions efficiently.

Business Context

Card fraud continues to be a significant threat in the banking and financial services industry, accounting for billions of dollars in loss yearly. Accurate identification of fraudulent transactions can help to mitigate financial loss and ensure consumer protection.

However, fraud detection is difficult because fraud cases are very rare (high class imbalance) and fraud attack patterns change over time. In practical terms, this assignment responds to this challenge by implementing Autoencoder-based models for detecting anomalies, which enables them to learn the normal behavior for non-fraudulent transactions and flag anomalies with no need for labelled data.

Dataset Overview

We have used [Kaggle's Credit Card Fraud detection dataset](#)

The dataset contains transactions made by credit cards in September 2013 by European cardholders.

This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.

It has 30 numerical features

- Time: Seconds elapsed between the transaction and the first transaction
- Amount: Transaction Amount
- Class: 1 [Fraud], 0 [Non-Fraud]
- V1 to V28: Principal Components obtained via PCA [Principal Component Analysis]. This is done to ensure confidentiality and reduce dimensionality

Data Preprocessing

We have performed the following techniques on our data for preprocessing. We explored the data to find out there were no missing records in any column.

- Standardization of amount and time
- The standard transactions to be isolated for training (unsupervised learning)
- Test set construction — containing both fraud and non-fraud examples
- Split the input data into train and test sets while retaining the same distribution of the target class
- Created Test and Train tensors using data loaders

Since fraudulent data is not used during training, class imbalance has no impact on the model learning phase. And in real-world scenarios the ratio of fraudulent to non-fraudulent transactions is also similar. So, to preserve the original imbalance and reduce overfitting [which could have happened if we oversampled fraud data], we have not handled the imbalance in the dataset. Instead used it as strong foundation for training our data.

Techniques Chosen

1. Simple Autoencoders

We will be using Autoencoders for Anomaly detection. It will comprise of

- Compresses the input into a smaller format (Encoder).
- Reconstructs it back to the original form (Decoder).

The model learns to reconstruct normal (non-fraud) transactions well.

Autoencoder without any previous knowledge is an ideal choice for fraud detection as this is an unsupervised learning model that will only learn the patterns if it has been seen before. If we train an autoencoder only on normal data (non-fraudulent transactions), the autoencoder becomes very good at reconstructing normal data.

However, since fraud transactions are different from normal transactions, the reconstruction on fraud transactions is poor, leading the reconstruction error to be higher. This error could serve as a consistent anomaly score. Therby providing us with the below advantages:

- Doesn't require labeled fraud data, which is scarce.
- It learns the underlying structure of normal transactions very well.
- High reconstruction errors on fraud cases makes it easy to set a threshold and flag anomalies.
- Compared to deeper or probabilistic models, simple autoencoders are easier to train and understand and also easy on the compute.

Thus, making it our first choice for this assignment.

Architecture

- Input Dimension is 30
- Encoder: Dense(32) $\rightarrow$ Dense(16, RELU) $\rightarrow$ Dense(8, RELU) [This is our bottleneck]
- Decode: Dense(8, RELU) $\rightarrow$ Dense(16, RELU), Dense(30, Linear) [To match the input dimension]
- We are using Mean Squared Error (MSE) Loss as the loss function. The MSE loss measures the squared differences between the input data and the model's predictions.
- The Optimizer used is Adam optimizer
- The learning rate is set to 0.001
- The model will be trained for 20 epochs and input will be processed in batches

2. Variational Autoencoder

Variational Autoencoders (VAEs) are effective for credit card fraud detection because they excel at anomaly detection and work well with imbalanced and unlabeled data, which are common in fraud detection scenarios. It consists of:

Encoder: Compresses the input into a latent space using two fully connected layers, producing two outputs—mean (μ) and log-variance (logvar) for each input sample.

Reparameterization: Samples a latent vector z from the learned distribution using μ and logvar, allowing backpropagation through stochastic variables.

Decoder: Reconstructs the input from the latent vector z using two fully connected layers and an output layer.

Forward Pass: Returns the reconstructed input along with μ and logvar to compute the total VAE loss (reconstruction + KL divergence).

Architecture

- Input Dimension is 30
- Encoder: Linear(32→64, RELU) , Linear(64→32, RELU) . The output will be mean vector and log-variance vector which would be used for sampling
- Reparametrizing Layer: To allow gradient backpropagation through the stochastic sampling process: $z = \mu + \sigma * \epsilon$, where $\epsilon \sim N(0,1)$ and $\sigma = \exp(0.5 * \logvar)$
- Decoder: Linear (latent_dim → 32, ReLU), Linear (32 → 64, ReLU), Linear (64 → 30) → Output layer
- To compute the loss for a Variational Autoencoder (VAE) we are combining two components: the reconstruction loss and the Kullback-Leibler (KL) divergence.
 - The reconstruction loss, calculated using mean squared error (MSE), measures how well the model reconstructs the input data.
 - The KL divergence regularizes the latent space by ensuring that the learned distribution (with mean μ and variance $\logvar$) approximates a standard normal distribution.
 - The total VAE loss is the sum of these two components, encouraging accurate data reconstruction while maintaining a smooth and meaningful latent space, which is crucial for tasks like anomaly detection in fraud detection.
- The Optimizer used is Adam optimizer
- The learning rate is set to 0.001
- The model will be trained for 10 epochs

Evaluation Methods

Our focus was to use an anomaly-based approach to compare the impact of the Simple Autoencoder and the Variational Autoencoder (VAE) model's performance on a credit card fraud dataset. In this unsupervised setup, since all the training was on non-fraud data, the models were assessed on their ability to reconstruct genuine transactions and raise a flag for anomalies.

- **Reconstruction Error:** After training, each input sample was passed through the model, and we computed the Reconstruction Error using Mean Squared Error (MSE) between the original input and the reconstructed output.
- **Anomaly Thresholding:** A transaction was classified as fraudulent if its reconstruction error exceeded a threshold determined either using the 95th/99th percentile of normal data errors or by optimizing the F1-Score on the validation/test set.
- **Weighted Precision:** Proportion of predicted frauds that are truly frauds
- **Recall (Sensitivity):** Proportion of actual frauds correctly identified
- **F1-Score:** Harmonic mean of precision and recall
- **ROC-AUC (Area Under ROC Curve):** Measures overall model discrimination between classes

Results:

Simple Autoencoders

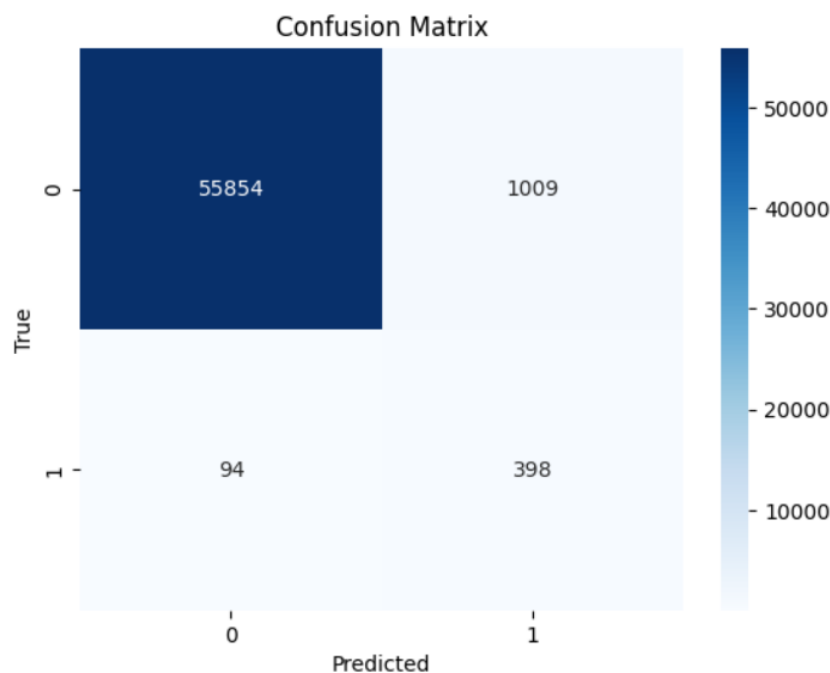
- **Anomaly Threshold** is the value above which the reconstruction error is considered high enough to classify the transaction as fraud. In this case, any transaction with a reconstruction error greater than **3.53673** is flagged as fraud.
- **Accuracy** is the proportion of correctly predicted instances (both non-fraud and fraud) out of the total instances. We have **98.1% accuracy** for our model which is great. But can also be due to imbalance in our dataset.
- **AUC (Area Under the Curve)** represents the performance of the model in terms of its ability to distinguish between fraud and non-fraud transactions. Our AUC score of 0.95 is excellent and shows that the model does a great job of distinguishing between fraud and non-fraud.

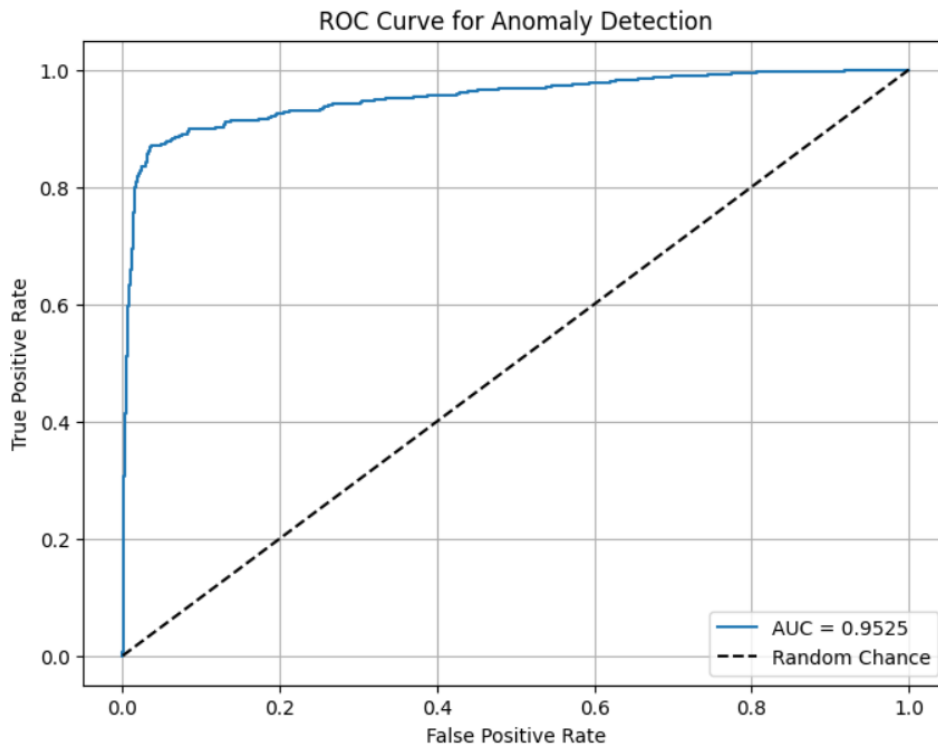
- **The F1 score is 0.42**, which indicates that while the model is able to correctly identify some fraudulent transactions, its ability to balance precision and recall is relatively poor. This could be due to the model being conservative in labeling transactions as fraud (resulting in more false positives).
- A **loss of 0.71789** means that, on average, the reconstruction error of the Autoencoder model is 0.71789.
- **Precision** is the proportion of true positives (fraud correctly identified) out of all instances predicted as fraud. In this case, **28.5% of the transactions flagged as fraud were fraud**.
- We have a highly imbalanced dataset, we will be using **Weighted Precision which is 0.99**. Weighted Precision averages the precision for each class (fraud and non-fraud), weighted by the number of true instances in each class. It means that overall, considering both classes, the model was 99.22% precise in its predictions.
- **Recall** is the proportion of true positives (fraud cases) correctly identified by the model out of all the actual fraud cases. Here, the model has a **recall of 80.7%, which is quite good, indicating that it successfully identifies a large proportion of fraudulent transactions**.

Classification Report

Threshold for anomaly detection: 3.5148				
	precision	recall	f1-score	support
0.0	1.00	0.98	0.99	56863
1.0	0.28	0.81	0.42	492
accuracy			0.98	57355
macro avg	0.64	0.90	0.70	57355
weighted avg	0.99	0.98	0.99	57355

Confusion Matrix





Variational Autoencoders

- The model achieves **high overall accuracy (97.84%)** and excellent performance on the majority (non-fraud) class.
- But its **actual precision** for detecting fraud (class 1) is low (**25.47%**), though The **weighted precision (99.18%)** is **good**.
- It manages a **high recall (79.07%)**—meaning it catches most fraud cases but with many false positives.
- Overall, the model is good at finding fraud but needs improvement in reducing false alarms.

Classification Report

Accuracy: 0.9784

Actual Precision: 0.2547

Precision Weighted: 0.9918

Recall: 0.7907

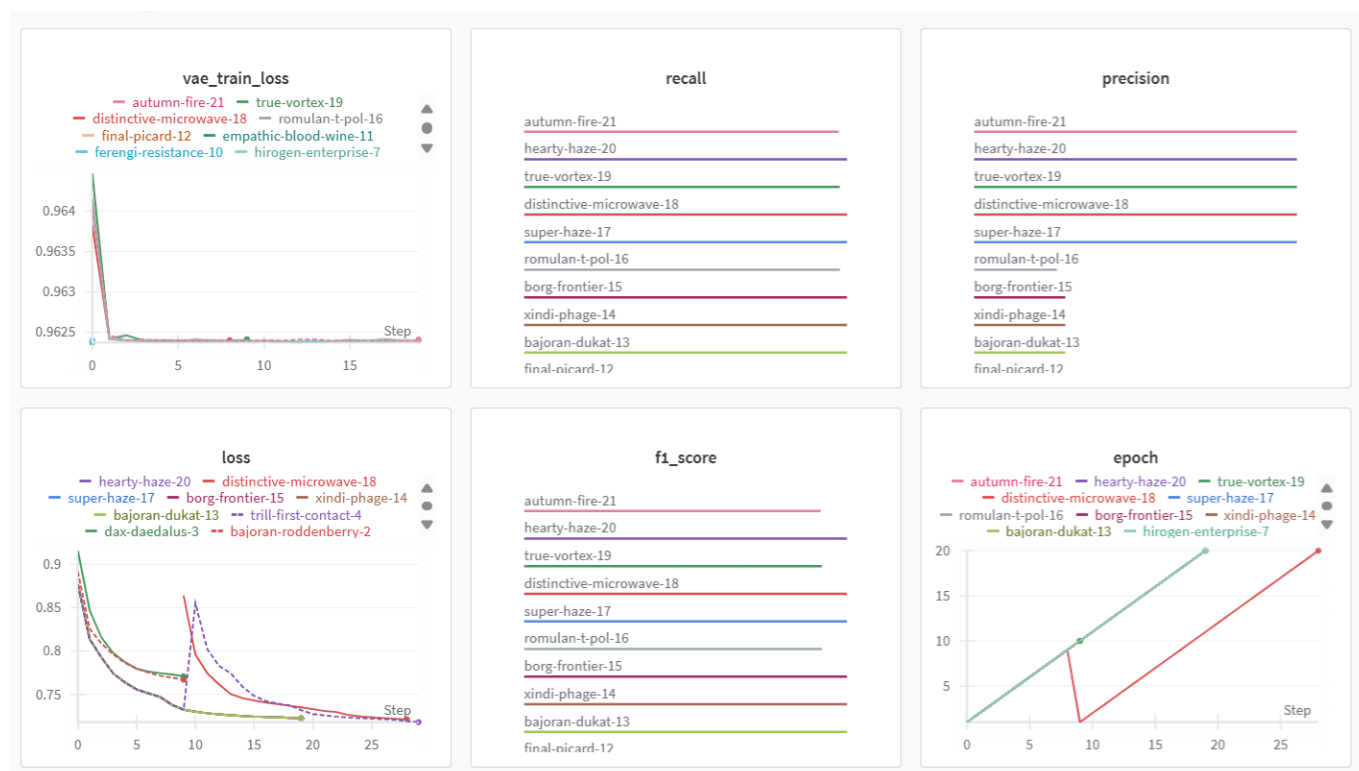
F1 Score: 0.3853

	precision	recall	f1-score	support
0	1.00	0.98	0.99	56863
1	0.25	0.79	0.39	492
accuracy			0.98	57355
macro avg	0.63	0.89	0.69	57355
weighted avg	0.99	0.98	0.98	57355

Confusion Matrix



We also tried to improve our model's precision using hyperparameter tuning. We have used our saved VAE model and hyper tuned it for different learning rates and batches. The evaluation metrics are somewhat like the original trained model. The improvements from tuning here are marginal but meaningful. We are validating the model's stability, robustness, and threshold sensitivity.



Conclusion

- Both models perform similarly in detecting fraud cases, with very close recall and F1 scores.
- VAE has slightly lower precision and accuracy compared to the simple autoencoder.
- The weighted precision remains high for both due to the overwhelming majority of non-fraud cases.

While VAE adds probabilistic modeling benefits and can generalize better with latent space learning, in this case, the simple autoencoder performs slightly better or equally well in fraud detection based on the provided metrics. We thereby infer that since our dataset is clean and consists simple data, Simple Autoencoder model is performing slightly better.

But in case of normal scenarios with noisy and complex data, The Variational Autoencoder model will perform better as VAE offers a probabilistic latent representation, which may help in other applications or long-term generalization.

Future scope

Hyperparameter tuning using more aggressive hyperparameters or switching architectures (like Conditional VAE or Transformer Autoencoders).